BTEUP EXAM NOTES | ENGLISH EDITION

Information Security

Unit 3: Cryptography, PKI and Secure Software Development

About these notes

Part A explains every topic in very simple English with real-life examples, so a beginner can understand it from zero.

Part B gives important exam questions with full 5-mark model answers, ready to write in the exam.

At the end: quick revision sheet, common mistakes, and a self-test.

Syllabus covered (Unit 3)

Basics of Cryptography, PKI, Security considerations while developing softwares.

Unit Overview and Contents

Simple idea: Cryptography is the science of **secret writing**: turning readable data into unreadable data so only the right person can read it. PKI is the system that proves **whose public key is whose**. And secure software development means writing programs so that attackers cannot break them. These three together protect data in storage, in transit and in applications.

Topic	Exam importance	Where in notes
Cryptography basics, classical ciphers	High	Part A: Topic 1 | Part B: Q1, Q9
Symmetric and Asymmetric cryptography	Very High	Part A: Topic 2 | Part B: Q2, Q3, Q4
Hash function, Digital signature	Very High	Part A: Topic 2 | Part B: Q5, Q6
PKI, Digital certificate	Very High	Part A: Topic 3 | Part B: Q7, Q8
Security in software development	Very High	Part A: Topic 4 | Part B: Q10, Q11, Q12

Contents

Part A: Easy Explanation	Part B: Important Questions with Answers
Topic 1: Basics of Cryptography Topic 2: Types of Cryptography, Hash, Digital Signature Topic 3: PKI Topic 4: Secure Software Development	Q1 Cryptography, terms and types (Very Likely) Q2 Symmetric key cryptography (Very Likely) Q3 Asymmetric key cryptography (Very Likely) Q4 Symmetric vs Asymmetric (Very Likely) Q5 Hash function (Likely) Q6 Digital signature (Likely) Q7 PKI and its components (Very Likely) Q8 Digital certificate (Likely) Q9 Caesar cipher, substitution and transposition (Possible) Q10 Security considerations in software development (Very Likely) Q11 SQL injection, XSS, buffer overflow (Likely) Q12 Secure SDLC (Possible)

How to study this unit (suggested time: 1 day)

1) Read Part A once to understand. 2) Read Part B answers and copy each answer on paper with the diagram. 3) Revise using the Quick Revision Sheet. 4) Attempt the Self-Test without looking.

PART A: Easy Explanation (Simple English)

Topic 1: Basics of Cryptography

Real-life example

You write a letter in a secret code that only your friend knows. Even if someone steals the letter, they cannot understand it. That is cryptography.

Definition

Cryptography is the science of protecting information by converting it into an unreadable form (encryption) so that only authorized persons can convert it back (decryption).

Basic terms

Term	Meaning
Plaintext	Original readable message
Ciphertext	Unreadable (encrypted) message
Encryption	Plaintext to ciphertext
Decryption	Ciphertext back to plaintext
Key	Secret value used by the algorithm
Algorithm (Cipher)	The method (steps) used to encrypt/decrypt
Cryptanalysis	Breaking a cipher without knowing the key

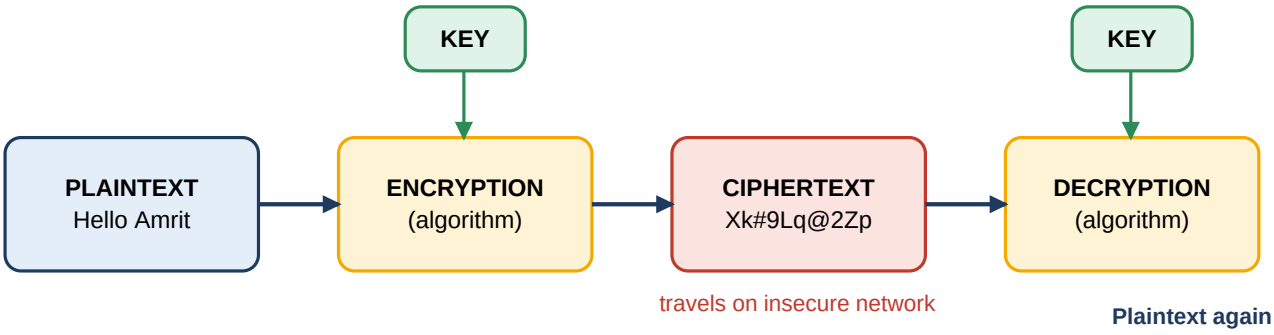


Figure 1: Encryption and decryption process

Goals of cryptography

- **Confidentiality:** only the right person can read.
- **Integrity:** message is not changed.
- **Authentication:** sender is genuine.
- **Non-repudiation:** sender cannot deny later.

Types of cryptography

- **Classical:** old methods, **substitution** (replace letters, e.g. Caesar cipher) and **transposition** (rearrange letters).
- **Modern: Symmetric key**, **Asymmetric key** and **Hash functions**.

Caesar cipher (shift = 3): each letter moves 3 places forward

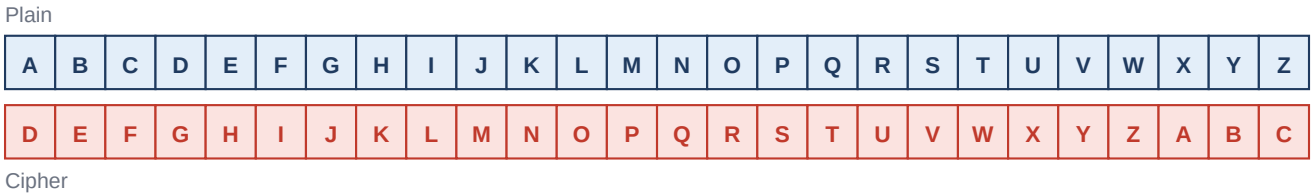


Figure 2: Caesar cipher (substitution cipher)

- **Substitution cipher:** each letter is replaced by another letter or symbol (Caesar, Monoalphabetic).
- **Transposition cipher:** letters stay the same but their **positions** change. Example: reading a message column by column.
- Classical ciphers are weak today because they can be broken by frequency analysis or brute force.

Topic 2: Symmetric, Asymmetric, Hash and Digital Signature

2(a) Symmetric Key Cryptography

Real-life example

A house key: the same key locks and unlocks the door. Anyone who has a copy of the key can open it. The big problem is giving the key safely to your friend.

Definition

In **symmetric key cryptography**, the **same secret key** is used for both encryption and decryption.

- **Very fast** and good for large data (files, disks, video).
- **Problem:** key distribution. How to share the secret key safely? Also, n users need many keys.
- **DES:** 64-bit block, 56-bit key, 16 rounds. Now weak. **3DES:** DES applied 3 times.
- **AES:** 128-bit block, key 128/192/256 bits (10/12/14 rounds). Current standard.
- Others: Blowfish, RC4 (stream cipher). Block cipher works on blocks, stream cipher on bit by bit.

2(b) Asymmetric Key Cryptography (Public Key)

Real-life example

A letterbox with a slot: anyone can drop a letter in (public key), but only the owner has the key to open the box (private key).

Definition

In **asymmetric key cryptography**, a **key pair** is used: a **public key** (shared with everyone) and a **private key** (kept secret). Data locked with one key can be opened only with the other.

- **Encryption:** sender uses the receiver's **public** key; receiver decrypts with his **private** key.
- **Digital signature:** sender signs with his **private** key; others verify with his **public** key.
- **RSA:** based on the difficulty of factoring large prime numbers; key size 2048 bits or more.
- **ECC:** gives the same security with much smaller keys. **Diffie-Hellman:** used to exchange a shared key safely.
- **Slower** than symmetric, so used for small data such as keys and signatures.

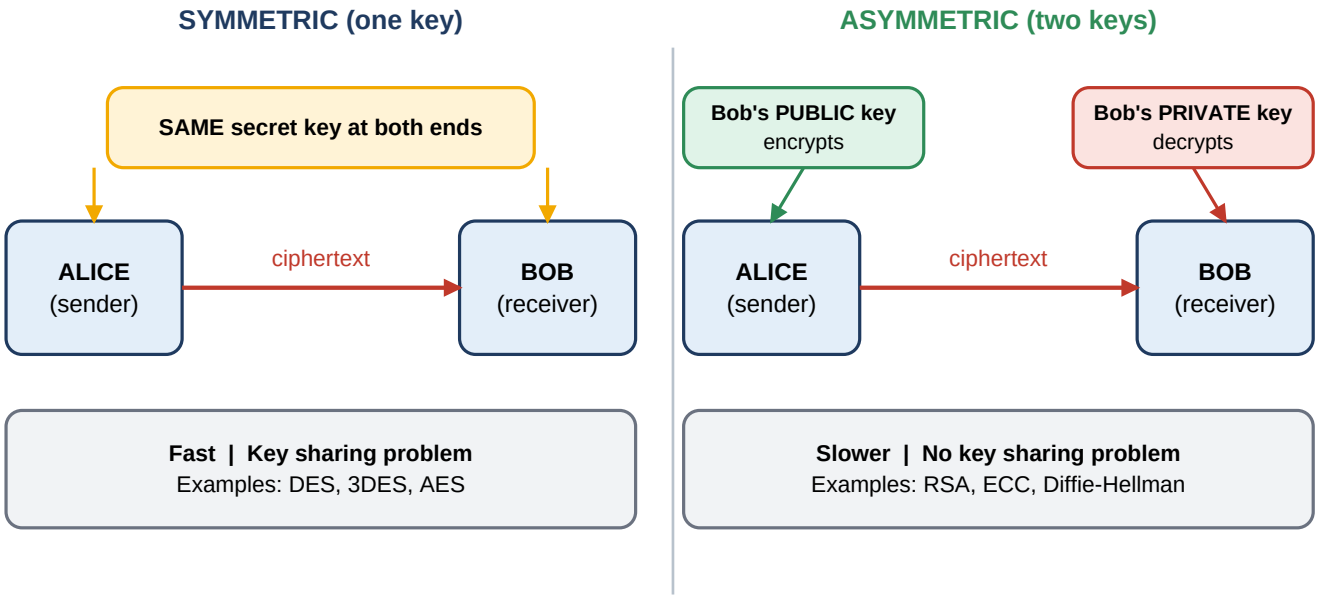


Figure 3: Symmetric vs Asymmetric cryptography

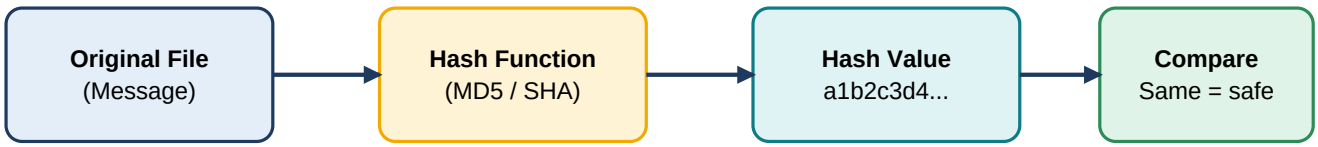
Hybrid approach (used in HTTPS)
Real systems use both: asymmetric cryptography to exchange a **session key** safely, then the fast symmetric key (AES) to encrypt the actual data.

2(c) Hash Function

Real-life example
A fingerprint: from a person you can get the fingerprint, but from the fingerprint you cannot rebuild the person. Even a tiny change in the person gives a different fingerprint.

Definition
A **hash function** converts data of any size into a **fixed-size value** (hash / message digest). It is a **one-way** function and uses **no key**.

- **Properties:** fixed output size, one-way (cannot be reversed), collision resistant (two inputs should not give the same hash), avalanche effect (1 bit change changes the hash completely), same input always gives same output.
- **Examples:** MD5 (128-bit, broken), SHA-1 (160-bit, broken), SHA-256 (256-bit, secure).
- **Uses:** integrity check, password storage, digital signatures.



If even 1 character changes, the hash value changes completely (Integrity check)

Figure 4: Hash function used for integrity checking

2(d) Digital Signature

Real-life example
It is like your handwritten signature plus a seal on a cheque: it proves that you signed it and that nobody changed the amount afterwards.

Definition
A **digital signature** is the hash of a message encrypted with the **sender's private key**. It gives **authentication, integrity and non-repudiation**.

- **Signing:** hash the message, then encrypt the hash with the private key.
- **Verifying:** receiver decrypts the signature with the sender's public key (Digest 1), hashes the received message (Digest 2) and compares.
- If both match, the message is genuine and unchanged.

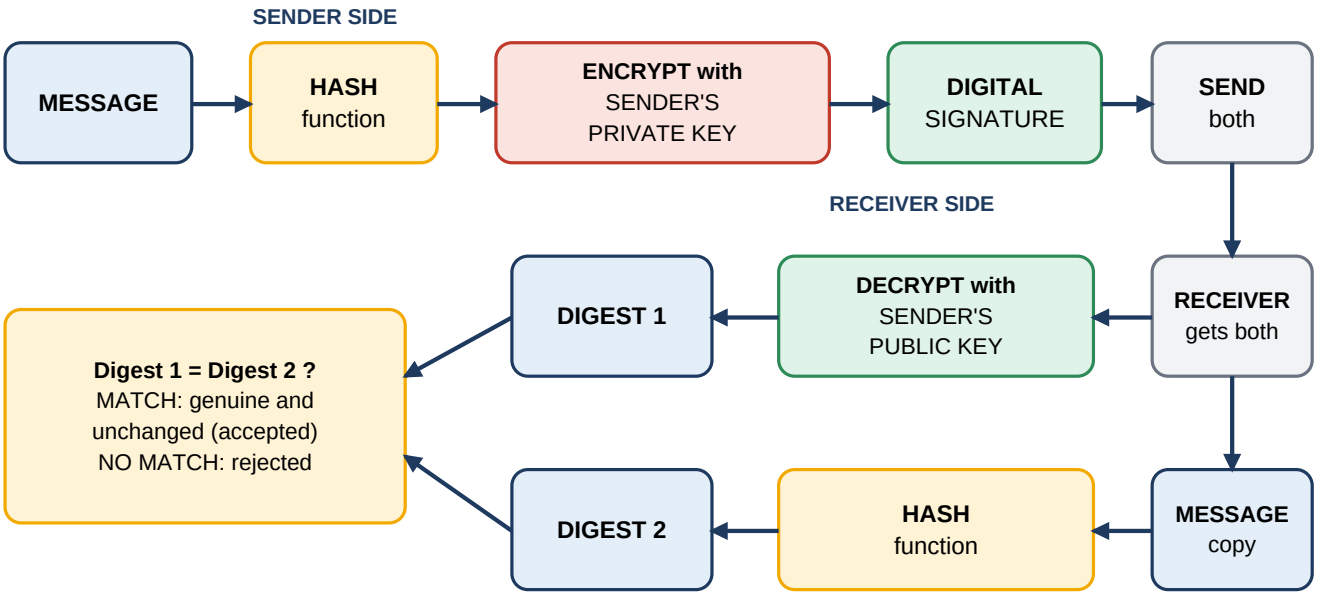


Figure 5: Digital signature: signing and verification

Topic 3: PKI (Public Key Infrastructure)

Real-life example

PKI is like the passport office. A passport proves who you are because a trusted government office checked your identity and signed it. In the same way, a trusted CA checks a person or website and signs a digital certificate.

Definition

PKI is the set of people, policies, hardware, software and procedures needed to create, manage, distribute, use, store and revoke **digital certificates** and public keys.

Why is PKI needed?

With public key cryptography, how do you know a public key really belongs to Bob and not to an attacker? PKI solves this by binding a public key to an identity using a certificate signed by a trusted authority.

Components of PKI

Component	Role
Certificate Authority (CA)	Trusted body that issues and signs digital certificates
Registration Authority (RA)	Verifies the identity of the applicant on behalf of the CA
Digital Certificate	Electronic document linking a public key to its owner (X.509)
Public / Private key pair	Keys of the user used for encryption and signing
Directory / Repository	Stores certificates and revocation lists
CRL and OCSP	Tell whether a certificate has been revoked

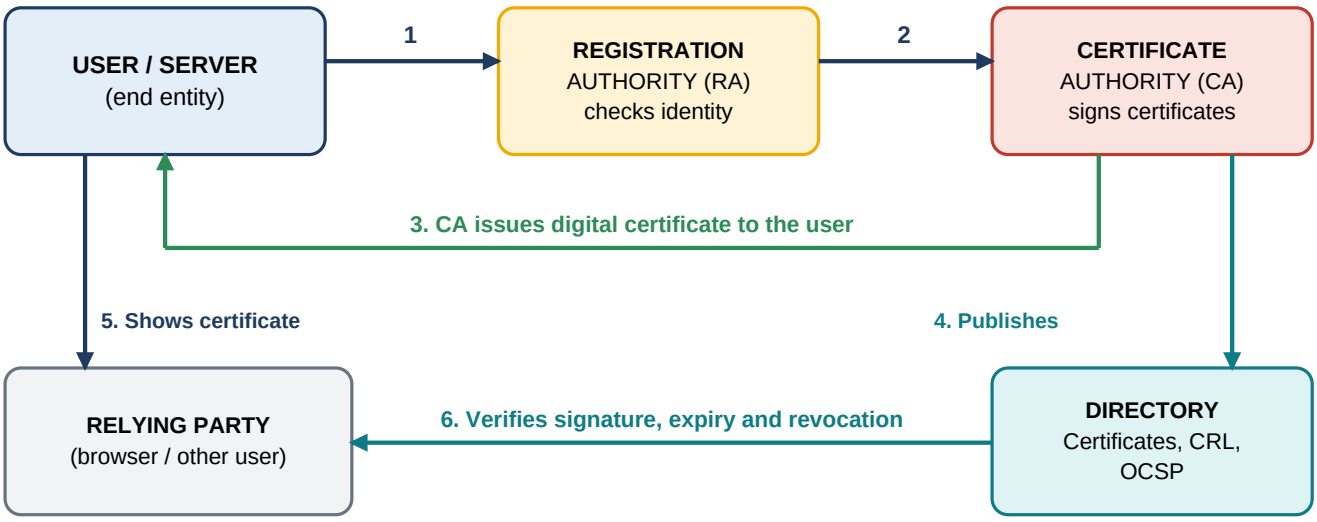


Figure 6: How PKI works

Certificate life cycle

- **1. Key generation:** user creates a public/private key pair.
- **2. Request (CSR):** user sends the public key and identity details to the RA/CA.
- **3. Verification:** RA checks the identity.
- **4. Issue:** CA signs and issues the certificate.
- **5. Use:** certificate is used in HTTPS, email, code signing, VPN.

- **6. Renewal or revocation:** renewed before expiry, or revoked if the private key is stolen.

Digital Certificate (X.509)

- It contains the owner's name, the owner's **public key**, validity dates, the issuer (CA) name, a serial number and the **CA's digital signature**.
- A browser trusts a certificate only if it is signed by a trusted CA, is not expired and the domain name matches.
- **Trust chain:** Root CA signs Intermediate CA, which signs the website certificate.

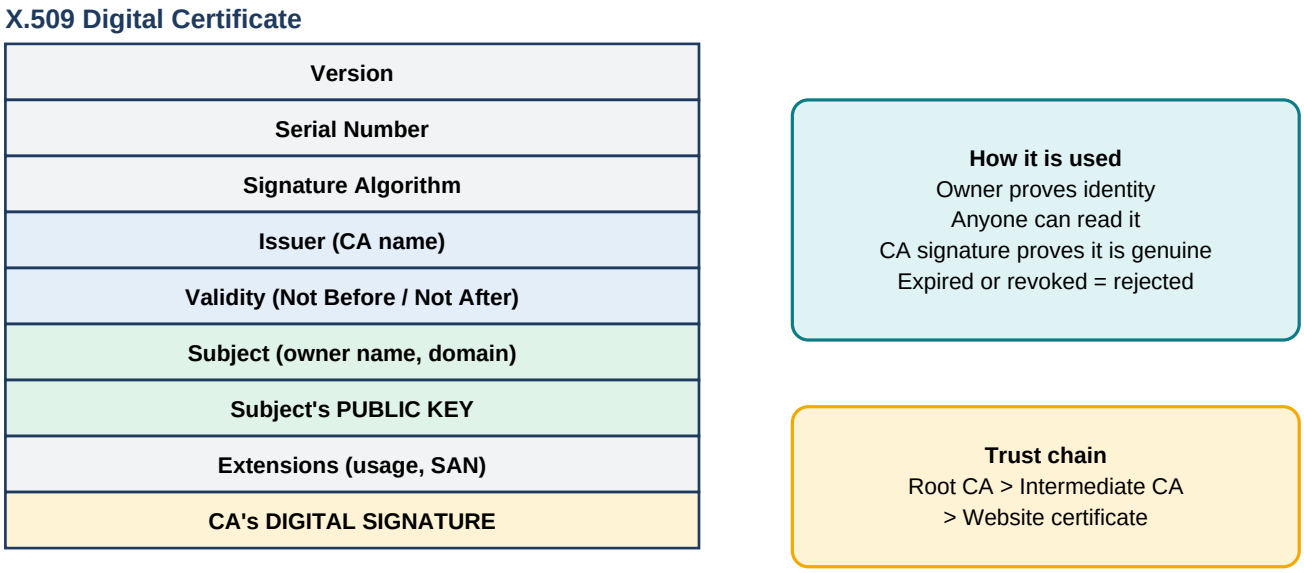


Figure 7: Structure of an X.509 digital certificate

Topic 4: Security Considerations while Developing Software

Real-life example

If a builder forgets to put locks and strong walls while constructing a house, adding security later is costly and weak. Software is the same: think about security **from the start**, not after the product is ready.

Definition

Secure software development means designing, coding and testing software in a way that reduces vulnerabilities and protects it from attacks.

Common software vulnerabilities

- **Buffer overflow:** a program writes more data than a buffer can hold and overwrites nearby memory.
- **SQL injection:** attacker enters SQL commands in an input field to read or change the database.
- **Cross-Site Scripting (XSS):** attacker injects a malicious script that runs in other users' browsers.
- **Weak authentication / hard-coded passwords** in the code.
- **Poor error handling:** detailed error messages reveal system information.
- **Insecure data storage:** passwords or data saved without encryption/hashing.
- **Outdated libraries** with known bugs.

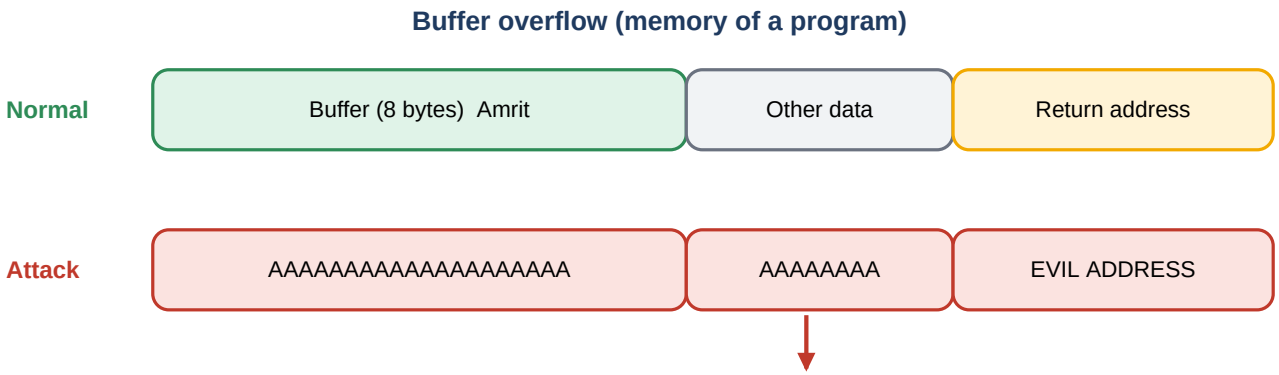


Figure 8: Buffer overflow attack

SQL injection example

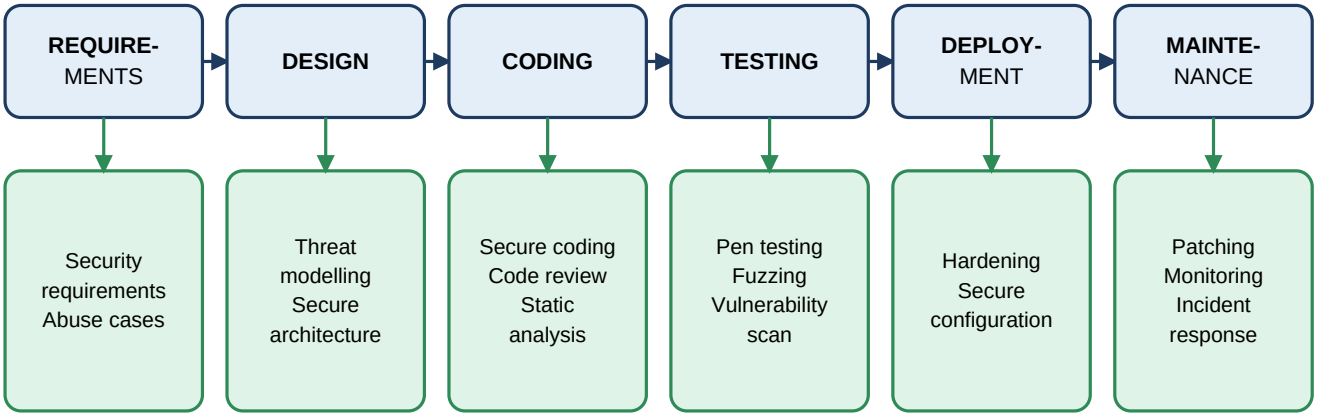
Query in code: `SELECT * FROM users WHERE name='[input]' AND pass='[input]'`
If the attacker types `' OR '1'='1` as input, the condition is always true and login is bypassed.
Fix: use parameterized queries (prepared statements) and input validation.

Secure coding practices (considerations)

- **Input validation:** never trust user input. Check type, length and format.
- **Least privilege:** program should run with minimum rights.
- **Secure authentication and session management:** strong passwords, hashing with salt, MFA, session timeout.
- **Encrypt sensitive data** at rest and in transit (HTTPS/TLS).
- **Safe error handling and logging:** show generic errors to users, log details privately.
- **Secure defaults, defence in depth, fail-safe:** use several layers of security; on failure, deny access.
- **Code review, testing and patching:** static analysis, penetration testing, fuzzing, regular updates.
- **Use trusted libraries** and follow standards such as **OWASP Top 10**.

Secure SDLC

Security is added in every phase of the Software Development Life Cycle.



Security activity in every phase (Secure SDLC)

Figure 9: Secure Software Development Life Cycle

PART B: Important Questions with Model Answers (English)

Priority: **Very Likely** = appears most often in BTEUP. **Likely** = good chance. **Possible** = can appear. Each answer follows the exam format: **Definition, Points, Diagram/Table, Conclusion**.

Q1. What is Cryptography? Explain its basic terms and types. — VERY LIKELY

Definition / Introduction: Cryptography is the science of protecting information by converting it into an unreadable form (encryption) so that only authorized persons can convert it back to readable form (decryption).

Points

- **Basic terms:** plaintext (original message), ciphertext (encrypted message), encryption, decryption, key (secret value), algorithm/cipher (method), cryptanalysis (breaking a cipher).
- **Goals:** confidentiality, integrity, authentication and non-repudiation.
- **Types:** (1) Symmetric key (one key), (2) Asymmetric key (public + private key), (3) Hash functions (no key).
- **Classical ciphers:** substitution (Caesar) and transposition.

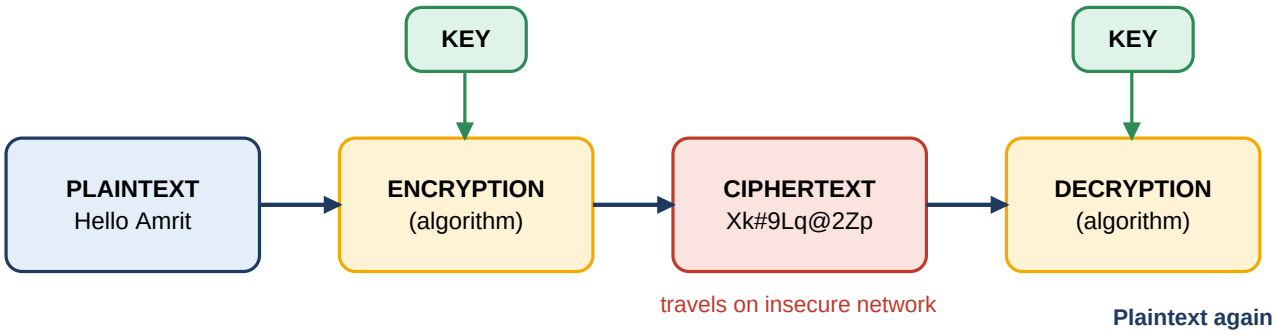


Figure: Encryption and decryption

Conclusion: Cryptography is the foundation of secure communication, e-commerce and data protection.

Q2. Explain Symmetric Key Cryptography with examples. VERY LIKELY

Definition / Introduction: In symmetric key cryptography, the same secret key is used for both encryption and decryption.

Points

- **Working:** the sender encrypts with the shared key; the receiver decrypts with the same key.
- **Advantages:** very fast, simple, suitable for large data.
- **Disadvantages:** key distribution problem; many keys needed for many users (n(n-1)/2); no non-repudiation.
- **Examples:** DES (56-bit key, 64-bit block, 16 rounds), 3DES, AES (128/192/256-bit key), Blowfish, RC4.

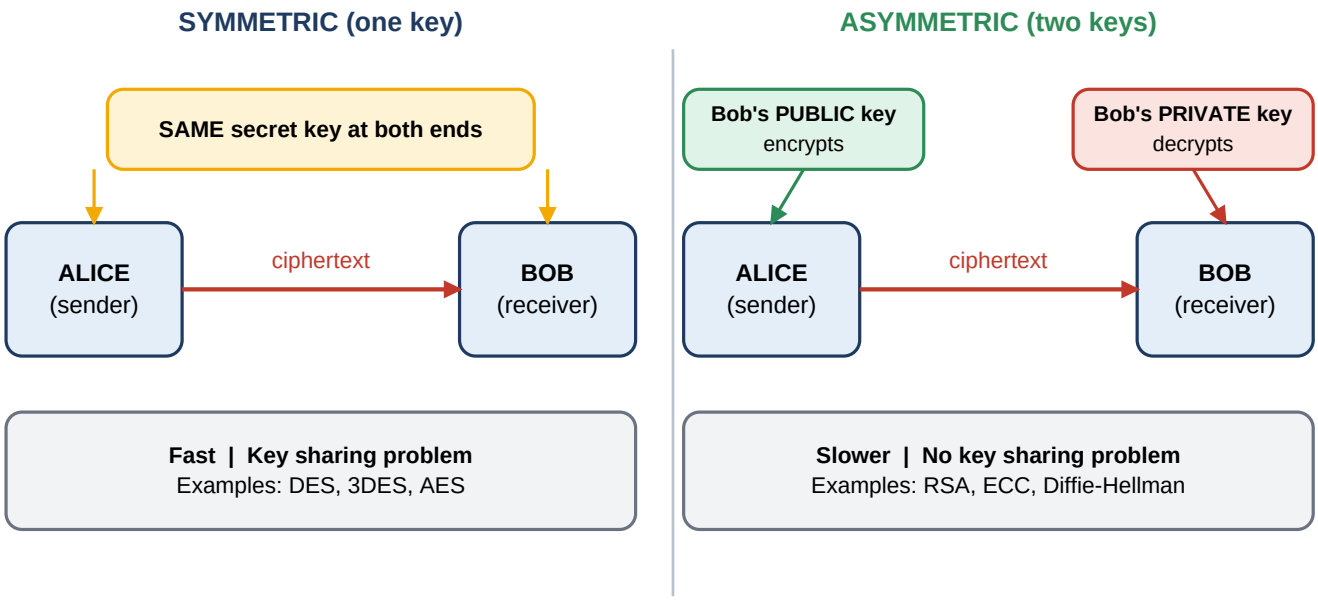


Figure: Symmetric and asymmetric cryptography

Point	DES	AES
Key size	56 bits	128 / 192 / 256 bits
Block size	64 bits	128 bits
Rounds	16	10 / 12 / 14
Security	Weak (outdated)	Strong (current standard)

Conclusion: Symmetric cryptography is fast and best for bulk data, but needs a safe method of sharing the key.

Q3. Explain Asymmetric Key Cryptography (RSA). VERY LIKELY

Definition / Introduction: Asymmetric key (public key) cryptography uses a pair of keys: a public key known to everyone and a private key kept secret. Data encrypted with one key can be decrypted only with the other.

Points

- **Confidentiality:** sender encrypts with the receiver's public key; only the receiver's private key can decrypt.
- **Digital signature:** sender signs with his private key; anyone can verify with his public key.
- **RSA** (Rivest, Shamir, Adleman) is based on the difficulty of factoring the product of two large primes. Key size 2048 bits or more.
- **Other algorithms:** ECC, ElGamal, Diffie-Hellman (key exchange).
- **Advantages:** no key sharing problem, supports signatures. **Disadvantage:** slow.

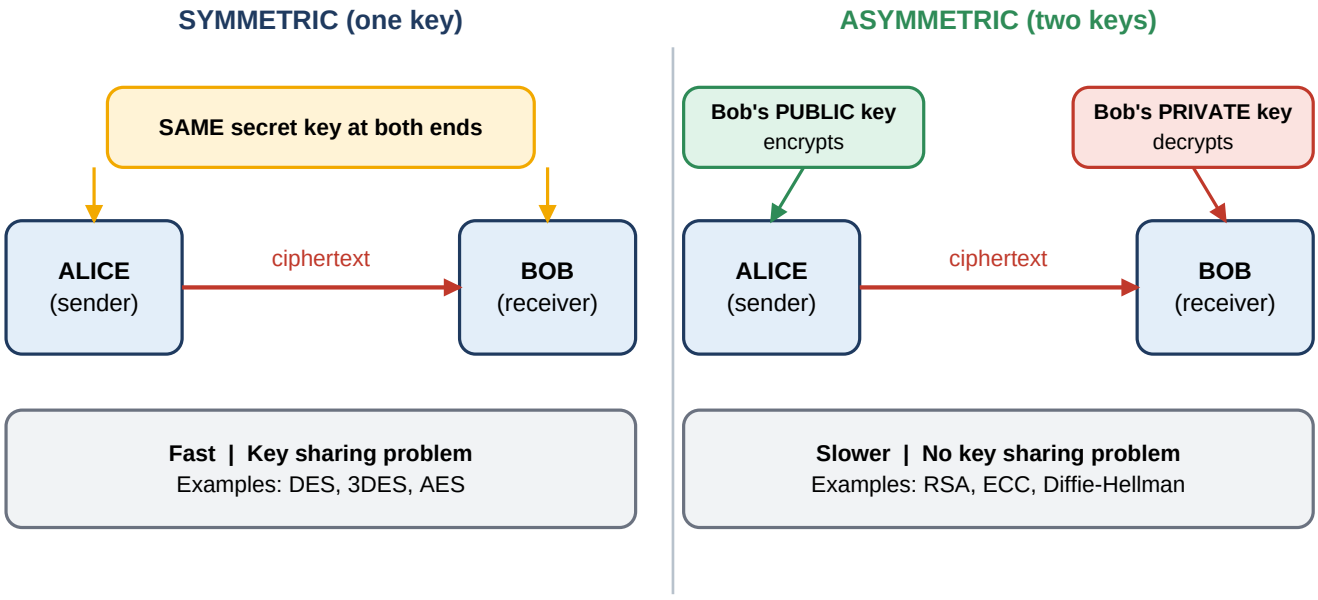


Figure: Symmetric and asymmetric cryptography

Conclusion: Asymmetric cryptography solves key distribution and enables digital signatures; it is used together with symmetric cryptography in practice.

Q4. Differentiate between Symmetric and Asymmetric key cryptography. VERY LIKELY

Definition / Introduction: Symmetric cryptography uses one shared secret key, whereas asymmetric cryptography uses a public-private key pair.

Points

- Both are used to provide confidentiality, but they differ in keys, speed and use as shown below.

Point	Symmetric	Asymmetric
Number of keys	One (same key)	Two (public + private)
Speed	Fast	Slow
Key sharing	Problem (secret key must be shared)	No problem (public key is open)
Best for	Large data	Small data, keys, signatures
Number of keys for n users	n(n-1)/2	2n
Examples	DES, 3DES, AES	RSA, ECC, Diffie-Hellman

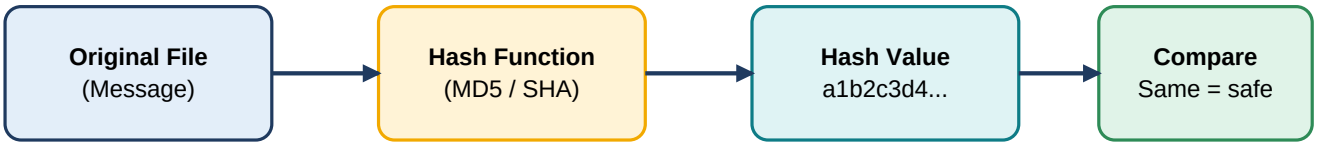
Conclusion: In practice both are combined (hybrid): asymmetric for key exchange and symmetric for data.

Q5. What is a Hash function? Explain its properties and uses. LIKELY

Definition / Introduction: A hash function takes input of any size and produces a fixed-size output called hash value or message digest. It is one-way and uses no key.

Points

- **Fixed output:** MD5 gives 128 bits, SHA-1 160 bits, SHA-256 256 bits.
- **One-way (pre-image resistance):** original data cannot be found from the hash.
- **Collision resistance:** it is very hard to find two inputs with the same hash.
- **Avalanche effect:** a tiny change in input changes the hash completely.
- **Deterministic:** the same input always gives the same hash.
- **Uses:** file integrity check, password storage, digital signatures, blockchain.



If even 1 character changes, the hash value changes completely (Integrity check)

Figure: Hash function and integrity check

Conclusion: Hash functions provide integrity and are a building block of digital signatures.

Q6. What is a Digital Signature? Explain how it works. LIKELY

Definition / Introduction: A digital signature is a value created by encrypting the hash of a message with the sender's private key. It provides authentication, integrity and non-repudiation.

Points

- **Signing:** sender calculates the hash (digest) of the message and encrypts it with his private key. This is the signature.
- The sender sends the message together with the signature.
- **Verification:** receiver decrypts the signature with the sender's public key to get Digest 1, and hashes the received message to get Digest 2.
- If Digest 1 = Digest 2, the message is genuine and unchanged; otherwise it is rejected.

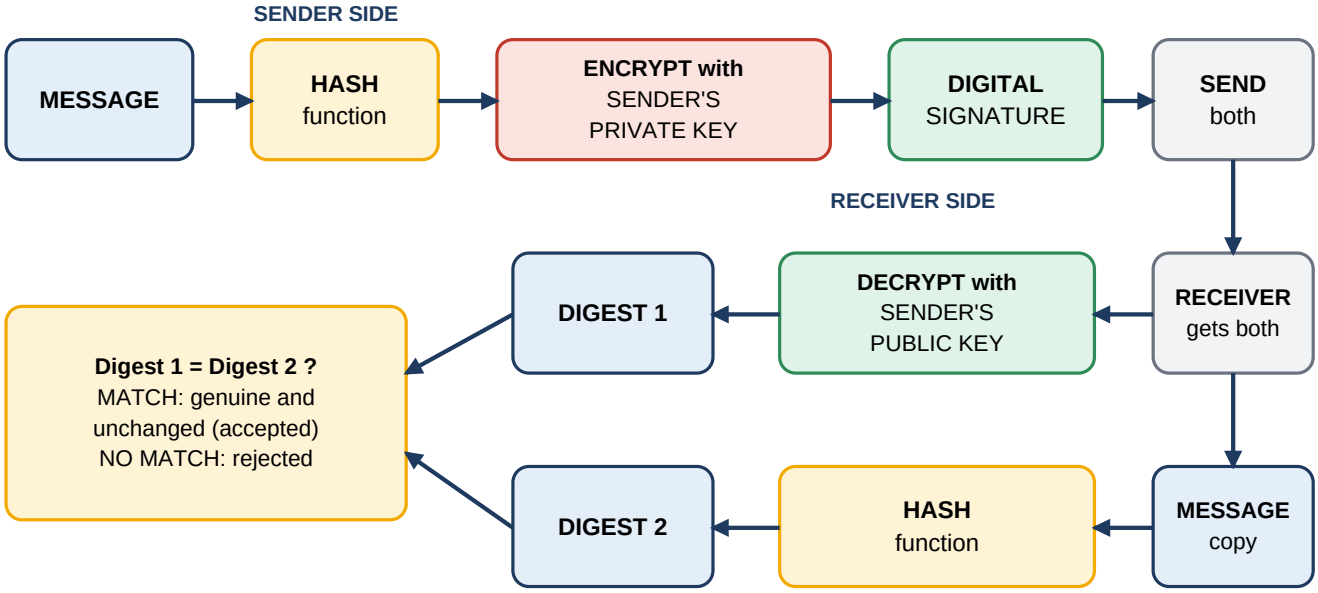


Figure: Digital signature process

Conclusion: Digital signatures are used in e-governance, online banking, software distribution and legal documents.

Q7. What is PKI? Explain its components and working. VERY LIKELY

Definition / Introduction: PKI (Public Key Infrastructure) is a framework of people, policies and technology used to create, manage, distribute and revoke digital certificates, so that public keys can be trusted.

Points

- **Certificate Authority (CA):** trusted body that issues and digitally signs certificates.
- **Registration Authority (RA):** verifies the identity of the applicant for the CA.
- **Digital certificate (X.509):** binds a public key to its owner.
- **Directory / repository:** stores certificates; **CRL / OCSP:** list or service for revoked certificates.
- **Working:** user requests a certificate, RA verifies identity, CA issues it, user presents it, and others verify the CA's signature, the validity period and the revocation status.

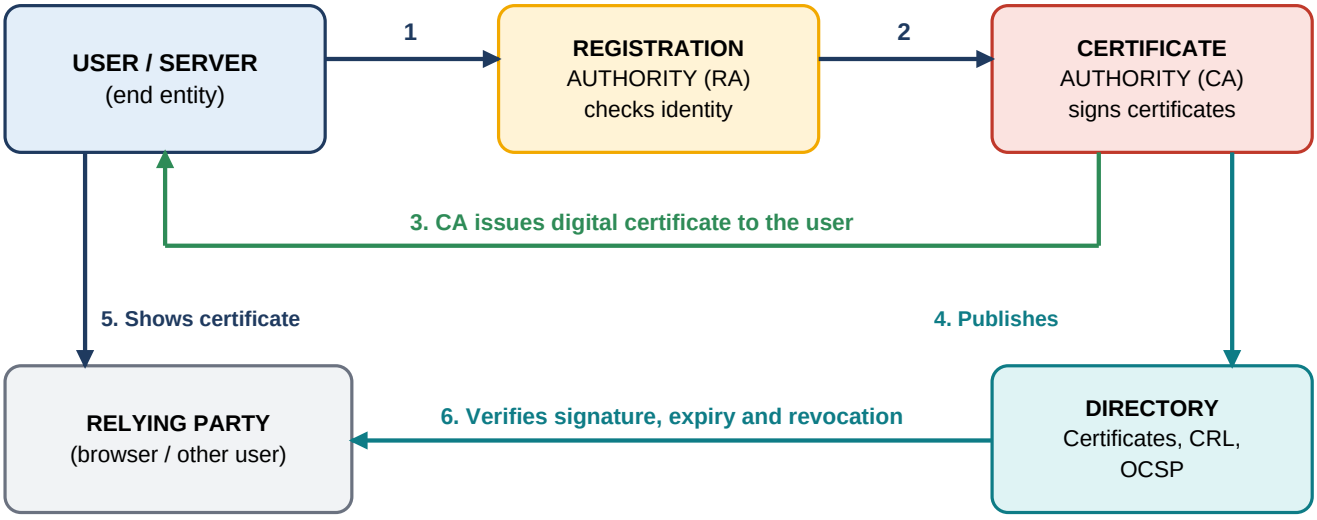


Figure: Working of PKI

Conclusion: PKI provides trust on the internet and supports HTTPS, secure email, VPN and digital signatures.

Q8. What is a Digital Certificate? Explain its contents. LIKELY

Definition / Introduction: A digital certificate is an electronic document, issued and signed by a Certificate Authority, that proves the ownership of a public key. The standard format is X.509.

Points

- **Contents:** version, serial number, signature algorithm, issuer (CA) name, validity period, subject (owner) name, subject's public key, extensions and the CA's digital signature.
- **Verification:** the browser checks that the CA is trusted, the signature is valid, the certificate is not expired or revoked, and the domain matches.
- **Trust chain:** Root CA, then Intermediate CA, then the end certificate.
- **Uses:** HTTPS websites, email security (S/MIME), code signing, VPN, smart cards.

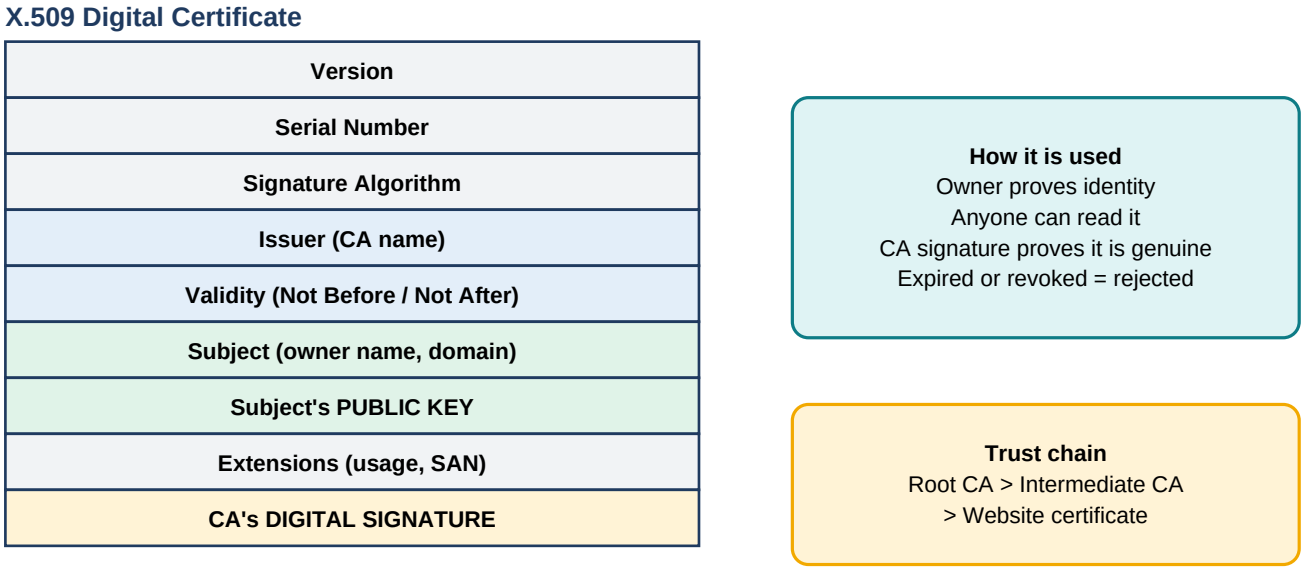


Figure: X.509 certificate structure

Conclusion: Digital certificates let users trust a public key without meeting the owner.

Q9. Explain Caesar cipher. What are substitution and transposition ciphers?

POSSIBLE

Definition / Introduction: Classical ciphers are old encryption methods. They are of two basic types: substitution and transposition.

Points

- **Substitution cipher:** each letter is replaced by another letter or symbol.
- **Caesar cipher:** each letter is shifted by a fixed number (key). With shift 3: A becomes D, B becomes E, and so on. Formula: C = (P + k) mod 26 and P = (C - k) mod 26.
- **Example:** HELLO with shift 3 becomes KHOOR.
- **Transposition cipher:** letters are not changed, only their positions are rearranged (for example, writing the text in rows and reading by columns).
- **Weakness:** Caesar cipher has only 25 possible keys, so brute force and frequency analysis break it easily.

Caesar cipher (shift = 3): each letter moves 3 places forward

Plain

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Cipher

HELLO -> KHOOR (H->K, E->H, L->O, L->O, O->R)

Figure: Caesar cipher with shift 3

Conclusion: Classical ciphers are not secure today, but they explain the basic ideas of modern cryptography.

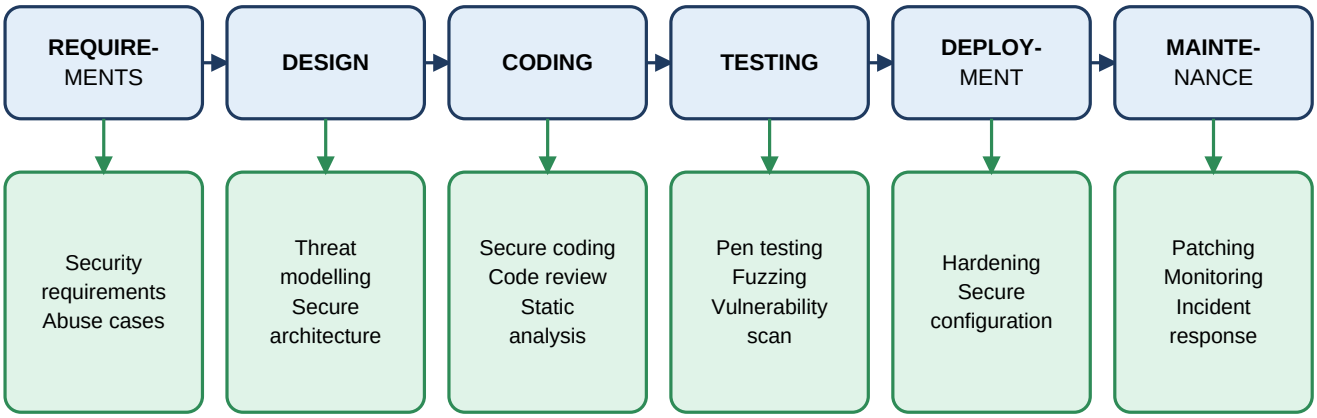
Q10. Explain the security considerations while developing software.

VERY LIKELY

Definition / Introduction: Secure software development means building software so that vulnerabilities are minimised and it can resist attacks. Security must be considered from the start of development.

Points

- **Input validation:** check all user input for type, length and format; never trust it.
- **Authentication and session management:** strong passwords, salted hashing, MFA, session timeout.
- **Least privilege:** run code with minimum rights.
- **Data protection:** encrypt sensitive data at rest and in transit; never hard-code passwords or keys.
- **Error handling and logging:** do not show internal details to users.
- **Defence in depth and secure defaults:** multiple layers; deny by default.
- **Testing and maintenance:** code review, static analysis, penetration testing, regular patches, updated libraries, follow OWASP Top 10.



Security activity in every phase (Secure SDLC)

Figure: Security in every phase of development

Conclusion: Building security in from the beginning is cheaper and safer than fixing problems after release.

Q11. Explain SQL Injection, XSS and Buffer Overflow vulnerabilities and their prevention. — LIKELY

Definition / Introduction: These are three of the most common software vulnerabilities, caused by poor handling of input.

Points

- **SQL injection:** attacker types SQL code (like `' OR '1'='1`) in a form to bypass login or read the database. **Prevention:** parameterized queries, input validation, least-privilege database account.
- **Cross-Site Scripting (XSS):** attacker injects a script into a web page that runs in other users' browsers and steals cookies. **Prevention:** output encoding, input validation, Content Security Policy, HttpOnly cookies.
- **Buffer overflow:** more data than the buffer size overwrites adjacent memory, such as the return address, letting the attacker run his code. **Prevention:** bounds checking, safe functions (strncpy instead of strcpy), stack canaries, ASLR, DEP.

Buffer overflow (memory of a program)

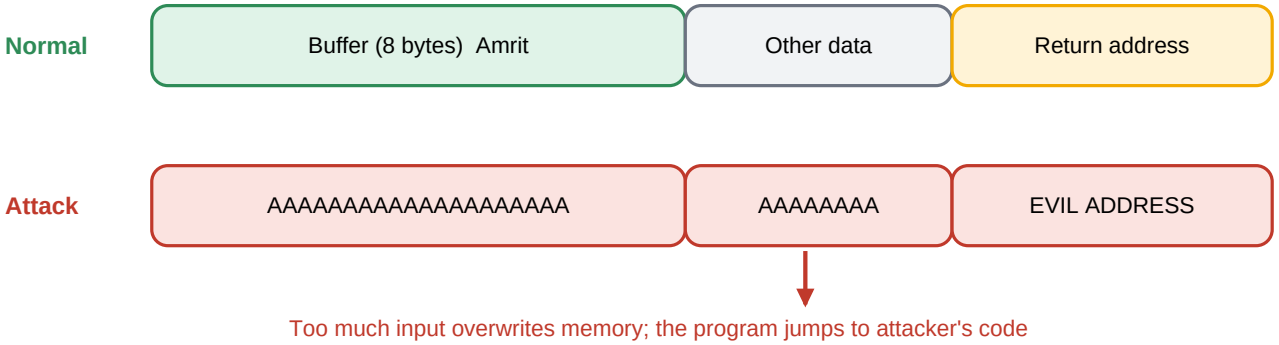


Figure: Buffer overflow

Attack	Target	Main fix
SQL injection	Database	Parameterized queries
XSS	Users' browsers	Output encoding, CSP
Buffer overflow	Program memory	Bounds checking, safe functions

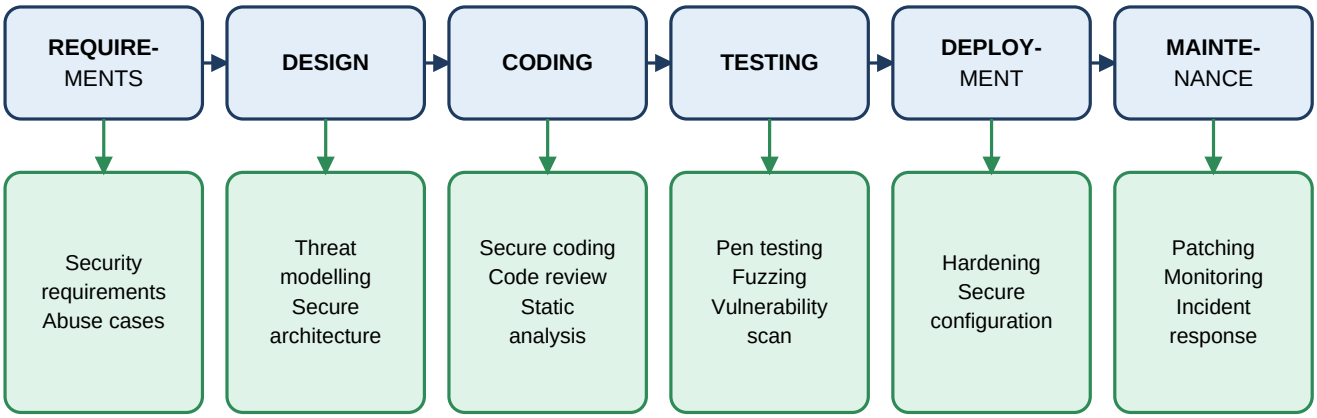
Conclusion: Validating input and using safe coding functions prevents most of these attacks.

Q12. Explain Secure SDLC (Software Development Life Cycle). POSSIBLE

Definition / Introduction: Secure SDLC integrates security activities into every phase of the software development life cycle, instead of testing for security only at the end.

Points

- **Requirements:** define security requirements and abuse cases.
- **Design:** threat modelling and secure architecture.
- **Coding:** secure coding standards, code review, static analysis.
- **Testing:** penetration testing, fuzzing, vulnerability scanning.
- **Deployment:** hardening and secure configuration.
- **Maintenance:** patching, monitoring and incident response.



Security activity in every phase (Secure SDLC)

Figure: Secure SDLC

Conclusion: Secure SDLC reduces vulnerabilities and the cost of fixing them.

Quick Revision Sheet (read the night before)

Term	Remember this
Cryptography	Plaintext to ciphertext using algorithm + key
Goals	Confidentiality, Integrity, Authentication, Non-repudiation
Caesar cipher	C = (P + k) mod 26. HELLO (k=3) = KHOOR
Substitution / Transposition	Change letters / Change positions
Symmetric	One key, fast. DES (56-bit), AES (128/192/256)
Asymmetric	Public + private key, slow. RSA, ECC, Diffie-Hellman
Encrypt for confidentiality	Receiver's PUBLIC key encrypts, PRIVATE key decrypts
Digital signature	Hash + sender's PRIVATE key. Verify with PUBLIC key
Hash	One-way, fixed size, no key. MD5 128, SHA-1 160, SHA-256
PKI	CA, RA, certificate, directory, CRL/OCSP
X.509 certificate	Owner, public key, issuer, validity, CA signature
Trust chain	Root CA > Intermediate CA > End certificate
SQL injection	Fix: parameterized queries
XSS	Script runs in other's browser. Fix: output encoding, CSP
Buffer overflow	Fix: bounds check, strncpy, ASLR, canaries
Secure SDLC	Security in all 6 phases: Req, Design, Code, Test, Deploy, Maintain

Memory tricks

Which key? Lock for secrecy = receiver's PUBLIC key. Sign for proof = my PRIVATE key.
PKI parts: C-R-C-D = CA, RA, Certificate, Directory.
Hash: O-F-C-A = One-way, Fixed size, Collision resistant, Avalanche.
Symmetric = Same key. Asymmetric = Another (different) key.

Common Mistakes Students Make

- Mixing up the keys: **encryption uses the receiver's public key**, but a **signature uses the sender's private key**.
- Saying a hash function uses a key or can be reversed. It is **keyless and one-way**.
- Confusing **encryption** (reversible with key) and **hashing** (not reversible).
- Confusing the roles of **CA** (issues certificate) and **RA** (verifies identity).
- Writing only the attack and not the **prevention** for SQL injection, XSS and buffer overflow.
- Not drawing diagrams (encryption flow, digital signature, PKI).

Self-Test (answer without looking, 5 marks each)

- Differentiate between symmetric and asymmetric cryptography with examples.
- Explain the working of a digital signature with a diagram.
- What is PKI? Explain its components.
- Encrypt the word HELLO using Caesar cipher with shift 3. Write the formula.
- What is SQL injection? How can it be prevented?

Self-Test Answers (check after writing)

1. Symmetric: one shared key, fast, key sharing problem, e.g. AES, DES. Asymmetric: public + private key pair, slow, no key sharing problem, supports signatures, e.g. RSA, ECC. Draw the table from Q4.

2. Sender hashes the message and encrypts the hash with his private key (signature) and sends both. Receiver decrypts the signature with the sender's public key (Digest 1), hashes the message (Digest 2) and compares. Equal means genuine and unchanged. Draw Figure 5.

3. PKI is a framework to create, manage, distribute and revoke digital certificates. Components: CA, RA, digital certificate, directory/repository, CRL/OCSP, key pairs.

4. Formula: C = (P + k) mod 26. H+3 = K, E+3 = H, L+3 = O, L+3 = O, O+3 = R. So HELLO becomes **KHOOR**.

5. SQL injection is when an attacker enters SQL code in an input field (e.g. ' OR '1'='1) to bypass login or access the database. Prevent it with parameterized queries, input validation and a least-privilege database account.

> **Done with Unit 3!**
> If you scored 4 out of 5 or more, move to **Unit 4**. If not, re-read the topics you missed in Part A.